

Array Pattern Notes – Increasing, Decreasing & Bitonic Subarrays

These are some of the most common interview problems on contiguous subarrays.

1. Longest Bitonic Subarray

Problem

Find the length of the longest contiguous subarray that first increases and then decreases.

Example

Input:
[12, 4, 78, 90, 45, 23]

Output:
5

Explanation:
[4, 78, 90, 45, 23]

Brute Force ($O(n^3)$)

Idea

Generate every possible subarray.

For each subarray,

- Check whether it is bitonic.
- Keep the maximum length.

Code

```
arr = [12, 4, 78, 90, 45, 23]
```

```
def bruteForce(arr):  
    ans = 1  
  
    def bitonic(sub):  
  
        phase = "inc"
```

```
    for i in range(1, len(sub)):

        if phase == "inc":

            if sub[i] >= sub[i - 1]:
                continue
            else:
                phase = "dec"

        if phase == "dec":

            if sub[i] <= sub[i - 1]:
                continue
            else:
                return False

    return True

for i in range(len(arr)):
    for j in range(i + 1, len(arr) + 1):

        if bitonic(arr[i:j]):
            ans = max(ans, j - i)

return ans

print(bruteForce(arr))
```

Complexity

```
Time   : O(n³)
Space  : O(1)
```

Optimized (O(n))

Idea

Create two arrays.

inc[]

Stores the length of the longest increasing subarray ending at index i.

Example

Array

1 2 3 2 1

inc

1 2 3 1 1

dec[]

Stores the length of the longest decreasing subarray starting at index i.

Array

1 2 3 2 1

dec

1 1 3 2 1

Formula

For every index,

Bitonic Length = $\text{inc}[i] + \text{dec}[i] - 1$

Subtract 1 because the peak element gets counted twice.

Code

```
arr = [12, 4, 78, 90, 45, 23]

def bitonic(arr):
    n = len(arr)
    inc = [1] * n
    for i in range(1, n):
        if arr[i] >= arr[i - 1]:
            inc[i] = inc[i - 1] + 1
```

```
dec = [1] * n

for i in range(n - 2, -1, -1):

    if arr[i] >= arr[i + 1]:
        dec[i] = dec[i + 1] + 1

ans = 1

for i in range(n):
    ans = max(ans, inc[i] + dec[i] - 1)

return ans

print(bitonic(arr))
```

Complexity

```
Time : O(n)
Space : O(n)
```

2. Longest Strictly Increasing Subarray

Problem

Find the length of the longest contiguous increasing subarray.

Example

Input

1 2 3 2 5 6 7

Output

3

Longest:

5 6 7

Idea

Maintain

```
current_length
```

If

```
arr[i] > arr[i-1]
```

extend the streak.

Otherwise,

start a new streak.

Code

```
arr = [1, 2, 3, 2, 5, 6, 7, 1]

def longestIncreasing(arr):

    cur = 1
    ans = 1

    for i in range(1, len(arr)):

        if arr[i] > arr[i - 1]:
            cur += 1
        else:
            cur = 1

        ans = max(ans, cur)

    return ans

print(longestIncreasing(arr))
```

Complexity

```
Time : O(n)
Space : O(1)
```

3. Longest Strictly Decreasing Subarray

Problem

Find the longest contiguous decreasing subarray.

Example

Input

9 8 7 6 2 5 4 3

Output

5

Longest

9 8 7 6 2

Idea

Exactly opposite of increasing.

If

```
arr[i] < arr[i-1]
```

extend the streak.

Else,

start a new streak.

Code

```
arr = [9, 8, 7, 6, 2, 5, 4, 3]

def longestDecreasing(arr):

    cur = 1
    ans = 1

    for i in range(1, len(arr)):
```

```
        if arr[i] < arr[i - 1]:
            cur += 1
        else:
            cur = 1

        ans = max(ans, cur)

    return ans

print(longestDecreasing(arr))
```

Complexity

Time : $O(n)$
Space : $O(1)$

4. Maximum Sum Bitonic Subarray

Problem

Find the maximum possible sum of any bitonic subarray.

Example

Input

1 3 5 4 2

Output

15

Subarray

1 3 5 4 2

Sum

15

Idea

Instead of storing **length**, store **sum**.

inc[]

Stores increasing sum ending at index i.

Array

1 3 5 4 2

inc

1

4

9

4

2

dec[]

Stores decreasing sum starting from index i.

dec

1

3

11

6

2

Formula

$\text{inc}[i] + \text{dec}[i] - \text{arr}[i]$

Subtract the peak once because it is counted twice.

Code

```
arr = [1, 3, 5, 4, 2]
```

```
def maxSumBitonic(arr):  
    n = len(arr)  
    inc = arr[:]  
    for i in range(1, n):  
        if arr[i] > arr[i - 1]:  
            inc[i] = inc[i - 1] + arr[i]  
    dec = arr[:]  
    for i in range(n - 2, -1, -1):  
        if arr[i] > arr[i + 1]:  
            dec[i] = dec[i + 1] + arr[i]  
    ans = 0  
    for i in range(n):  
        ans = max(ans, inc[i] + dec[i] - arr[i])  
    return ans  
  
print(maxSumBitonic(arr))
```

Complexity

Time : $O(n)$
Space : $O(n)$

5. Check if an Array is Bitonic

Problem

Determine whether an array first increases and then decreases.

Example

Input

1 2 5 7 6 4 2

Output

True

Idea

Use two phases.

Initially,

Increasing

When the first decrease appears,

switch to

Decreasing

If an increase occurs again,

return

False

Otherwise,

return

True

Code

```
arr = [1, 2, 5, 7, 6, 4, 2]

def isBitonic(arr):
    phase = "inc"
    for i in range(1, len(arr)):
```

```
    if phase == "inc":
        if arr[i] > arr[i - 1]:
            continue
        else:
            phase = "dec"

    if phase == "dec":
        if arr[i] < arr[i - 1]:
            continue
        else:
            return False

    return True

print(isBitonic(arr))
```

Complexity

```
Time   : O(n)
Space  : O(1)
```

Comparison Table

Problem	Main Idea	Extra Space	Time
Longest Increasing Subarray	Current streak	O(1)	O(n)
Longest Decreasing Subarray	Current streak	O(1)	O(n)
Longest Bitonic Subarray	inc[] + dec[] lengths	O(n)	O(n)
Maximum Sum Bitonic Subarray	inc[] + dec[] sums	O(n)	O(n)
Check Bitonic Array	Two-phase traversal	O(1)	O(n)

Pattern to Remember

Longest problems

Store **length**.

```
current_length  
or  
inc[]  
dec[]
```

Maximum Sum problems

Store **sum**.

```
incSum[]  
decSum[]
```

Validation problems

Store the **current phase**.

```
Increasing  
  
↓  
  
Decreasing
```

Interview Trick

Whenever you encounter a contiguous array problem, ask yourself:

1. Do I need the **length**?
2. Do I need the **sum**?
3. Do I only need to validate a property?

Based on the answer:

- Length → Maintain streak or DP length array.
- Sum → Maintain running sum or DP sum array.
- Validation → Use state/phase traversal.

This simple pattern covers many interview questions on increasing, decreasing, mountain, and bitonic subarrays.